

DÉPARTEMENT INFORMATIQUE - IUT 2 GRENOBLE



Année Universitaire 2025-2026

MÉMOIRE D'ALTERNANCE

DÉVELOPPEMENT ET FINALISATION DE L'INDISS PLUS LAUNCHER

CORYS



Présenté par

Lili Goy

3^e Année Parcours A

Jury

IUT : Mme Chollat-Namy

IUT : Mme Dupuy-Chessa

Société : M. Cariteau

Déclaration de respect des droits d'auteurs

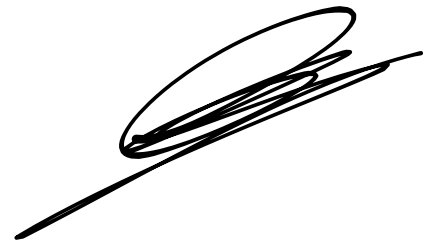
Par la présente, je déclare être la seule auteure de ce rapport et assure qu'aucune autre ressource que celles indiquées n'ont été utilisées pour la réalisation de ce travail. Tout emprunt (citation ou référence) littéral ou non à des documents publiés ou inédits est référencé comme tel et tout usage à un outil doté d'IA a été mentionné et sera de ma responsabilité.

Je suis informée qu'en cas de flagrant délit de fraude, les sanctions prévues dans le règlement des études en cas de fraude aux examens par application du décret 92-657 du 13 juillet 1992 peuvent s'appliquer. Elles seront décidées par la commission disciplinaire de l'UGA.

À Grenoble

Le 22/06/2026

Signature

A handwritten signature in black ink, consisting of a large, stylized loop followed by several horizontal strokes.

Remerciements

Je remercie tout d'abord l'entreprise Corys de m'avoir permis de réaliser ma troisième année de BUT en alternance, et plus particulièrement mes tuteurs, Monsieur Cariteau et Monsieur Durand de m'avoir accompagné dans mes missions et d'avoir répondu à toutes mes interrogations.

Je remercie vivement Monsieur Durand de m'avoir aidée quand je bloquais dans le projet et d'avoir débattu avec moi des différentes façons de coder les fonctionnalités que nous souhaitions. Ainsi que de m'avoir fait découvrir le Clean Code et d'autres ouvrages sur le sujet de l'informatique.

Je remercie également l'IUT pour leurs enseignements et leur accompagnement dans le processus d'obtention du contrat d'alternance. Je remercie aussi sincèrement ma tutrice, Madame Chollat-Namy, pour précieux ses conseils et ses corrections.

Enfin je remercie mes proches pour leur aide, leurs relectures et corrections.

Sommaire

I. LE PROJET INDISSPLUS LAUNCHER	7
I.1 CONTEXTE	7
I.2 EXISTANT	7
<i>L'OTS Manager</i>	7
<i>L'IndissPlus Launcher</i>	9
I.3 OBJECTIFS	10
<i>Partie édition</i>	10
<i>Partie exécution</i>	10
<i>Restant à faire</i>	11
I.4 CONTRAINTES ET RISQUES	11
II. ENVIRONNEMENT DE TRAVAIL	12
II.1 ARCHITECTURE DE L'APPLICATION	12
II.2 METHODES DE TRAVAIL	13
III. REFACTORING	16
III.1 APPLICATION DES CONSEILS DU CLEAN CODE	16
III.2 STEP TARGETSERVICE (ET LE PATTERN OBSERVER)	16
IV. DEVELOPPEMENT DE NOUVELLES FONCTIONNALITES	19
IV.1 AJOUTS DANS LE CYCLE D'EXECUTION DE SEQUENCE	19
IV.2 AJOUT D'UN MODE EXECUTION	20
IV.3 MODE EXECUTION : SEQUENCES	21
<i>Conception</i>	21
<i>Réalisation</i>	24
IV.4 MODE EXECUTION : ORDINATEURS	26
<i>Conception</i>	26
<i>Réalisation</i>	27
V. CONCLUSION	29
GLOSSAIRE	31
BIBLIOGRAPHIE	34

Table des Figures

Figure 1: Diagramme BPMN* couvrant le processus complet menant à l'exécution d'une séquence	8
Figure 2 : Interface d'édition de séquence depuis l'OTS Manager	9
Figure 3 : Interface de suivi d'exécution depuis l'OTS Manager.....	9
Figure 4 : Schéma des dépendances du projet IndissPlus Launcher, réalisé par M. Durand	12
Figure 5 : Schéma explicatif de l'architecture MVVM [4]	13
Figure 6 : Tableau Kanban du sprint actuel	14
Figure 7 : Diagramme de Gantt du planning rétrospectif	15
Figure 8 : Persona d'un instructeur type	20
Figure 9 : Maquettes des rapports d'exécutions.....	22
Figure 10 : Maquette de suivi d'exécution des étapes avec affichage dans la fenêtre de séquence	23
Figure 11 : Maquette de suivi d'exécution des étapes avec affichage dans le panneau latéral ...	23
Figure 12: Diagramme des classes StepResult et ComputerResult	25
Figure 13: Résultat du suivi d'exécution depuis le point de vue des étapes.....	26
Figure 14 : Maquettes de suivi d'exécution depuis la fenêtre des ordinateurs	27
Figure 15 : Diagramme de la classe StepResultPerComputer	27
Figure 16 : Résultat du suivi d'exécution depuis le point de vue des ordinateurs.....	28

Introduction

Je réalise mon alternance dans la société Corys, entreprise française à portée internationale [1]. L'entreprise est spécialisée dans le développement d'applications et de simulateurs, opérant dans deux secteurs distincts : le transport ferroviaire (trains, tramways), d'une part, et l'énergie, d'autre part. Dans ce second secteur, Corys intervient notamment dans le nucléaire en produisant des simulateurs et des jumeaux numériques pour les centrales EDF, ainsi que dans les industries de procédés*, un domaine regroupant notamment les secteurs de la pétrochimie, du raffinage, du traitement de l'eau, de l'agroalimentaire, mais également, de plus en plus, les secteurs des énergies renouvelables. C'est un domaine très polyvalent qui requiert un outil tout aussi polyvalent.

L'alternance s'effectue au sein du service dédié au secteur des industries de procédés. Les projets de ce service s'articulent principalement autour du logiciel IndissPlus, utilisé pour le développement de simulateurs de procédés industriels. L'équipe de développement, constituée de cinq développeurs aux parcours variés, travaille simultanément sur plusieurs projets visant à améliorer et compléter l'écosystème IndissPlus, à partir duquel l'équipe de production construit les simulateurs livrés aux entreprises clientes.

Les simulateurs construits par l'équipe de production sont avant tout destinés à la formation des opérateurs des entreprises clientes, dans le cadre de ce que l'on appelle des Operator Training Simulators (OTS). À l'image des pilotes d'avion qui s'entraînent sur simulateur avant de prendre les commandes d'un appareil réel, les opérateurs s'exercent sur ces simulateurs avant d'intervenir sur les installations industrielles réelles. Cet usage de formation représente environ 95 % de l'activité liée aux simulateurs développés par Corys, les 5 % restants correspondant à des études de faisabilité menées en amont de la construction de nouvelles installations ou de la mise en place de nouveaux procédés.

Dans ce contexte, la mission centrale de mon alternance est la réalisation d'un lanceur pour l'écosystème IndissPlus, l'IndissPlus Launcher, à destination notamment des instructeurs des entreprises clientes pour faciliter la mise en place des sessions de formation des opérateurs. Cette application joue un rôle d'orchestrateur, elle permet à l'instructeur de programmer une séquence d'actions à exécuter automatiquement sur les postes de la salle, puis de lancer cette séquence simultanément sur l'ensemble des machines concernées, sans avoir à intervenir manuellement sur chacune d'elles.

Ce rapport présente dans un premier temps le contexte du projet et ses objectifs. Il décrit ensuite l'environnement technique et humain dans lequel le projet s'inscrit. Les réalisations techniques sont ensuite abordées en deux temps : une première partie portant sur le refactoring de l'existant, suivie d'une partie détaillant les nouveaux développements conduits au cours de l'alternance.

I. Le projet IndissPlus Launcher

Cette partie présente le projet IndissPlus Launcher dans son ensemble : son contexte, l'existant dont il est issu, les objectifs fixés pour l'alternance, ainsi que les contraintes et risques identifiés tout au long du développement.

I.1 Contexte

L'écosystème IndissPlus est constitué de plusieurs applications :

- IndissPlus Builder (utilisé par l'équipe de production), qui permet de construire des simulateurs reproduisant les procédés chimiques utilisés dans les usines clientes. Ces simulateurs sont construits à partir de bibliothèques de composants de processus et de thermodynamique standard, ou de bibliothèques développées sur mesure pour les clients.
- IndissPlus Runtime, qui permet d'exécuter ces simulateurs et de gérer la vitesse de la simulation.
- IndissPlus Master, qui permet de lancer des simulateurs complexes en les séparant en sous-instances et en répartissant ces instances sur plusieurs machines.
- Competence Plus*, qui permet d'organiser des sessions de formation en ligne ou non (Competence Plus Cloud et On-Premise).

Ces applications servent principalement aux entreprises clientes à organiser des sessions de formations pour leurs opérateurs. Elles ont été historiquement organisées sur site, dans des salles informatiques. Ces sessions sont organisées par un instructeur et regroupe plusieurs opérateurs. Ces derniers se connectent sur les postes de la salle et suivent un scénario d'événements élaboré par l'instructeur, qu'il s'agisse de situations de routine ou d'urgence, afin de vérifier leur capacité à intervenir, ensuite, sur les machines réelles et de se familiariser avec ces différentes situations.

Avant chaque session, l'instructeur doit configurer l'ensemble des postes, une opération qui peut se révéler longue et fastidieuse selon le nombre de machines concernées, chaque étape devant être répétée individuellement sur chaque ordinateur.

Pour alléger cette tâche répétitive, Corys propose un logiciel supplémentaire à son écosystème. Celui-ci permet aux instructeurs de programmer les actions à exécuter sur les postes de la salle, puis de lancer l'exécution de cette séquence sur chacune des machines concernées. C'est cette application, l'IndissPlus Launcher, qui constitue le cœur de la mission de l'alternance.

Le Launcher* trouve également des usages en dehors des sessions de formation, notamment pour orchestrer le lancement de simulateurs complexes, nécessitant le démarrage et la synchronisation de plusieurs applications ainsi qu'une liaison à d'autres applications tierces.

I.2 Existant

L'OTS Manager

Corys propose depuis une dizaine d'années une application nommée OTS Manager*, permettant de créer des configurations contenant les séquences d'actions à exécuter et les ordinateurs sur lesquels elles seront lancées. L'utilisateur peut y éditer les séquences créées sous forme de listes chaînées d'étapes, les « steps* », disponibles en plusieurs sous-types (Wait, Execute, Start

IndissPlus, Stop, Stop IndissPlus...), afin de programmer au mieux les actions souhaitées. L'OTS Manager permet également d'ajouter des ordinateurs à la configuration et d'en paramétrer les informations afin d'établir la connexion entre le poste instructeur et le poste opérateur. Le processus couvrant l'édition d'une séquence et menant à son exécution est détaillé dans le schéma suivant.

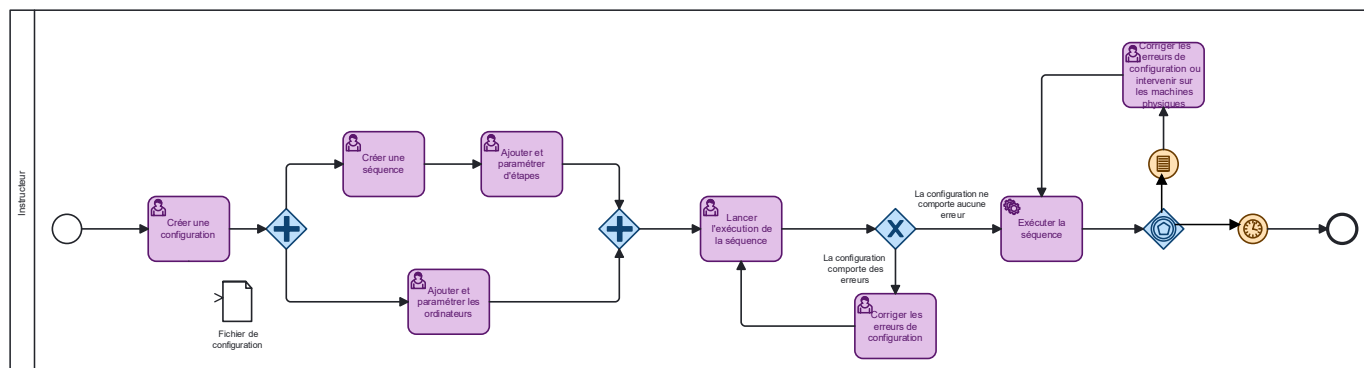


Figure 1: Diagramme BPMN* couvrant le processus complet menant à l'exécution d'une séquence

L'OTS Manager permet également de mettre en place toute la gestion des sessions de formation. Soit la gestion des comptes opérateurs, la gestion des applications utilisées par les instructeurs pour surveiller la formation et la sauvegarde dans une base de données locale.

Cette application est actuellement utilisée par plusieurs entreprises afin de mener à bien leurs sessions de formation. Néanmoins, plusieurs facteurs ont mené au lancement du développement d'une nouvelle application de lanceur. Tout d'abord le déploiement prochain de Compétence Plus, qui permettra la gestion de sessions de formation en ligne et remplacera l'OTS Manager sur site. Ensuite la sortie de la nouvelle version d'IndissPlus qui revoit complètement son interface graphique pour en proposer une plus moderne, le renouveau dont l'OTS Manager, qui présentait une interface très rudimentaire, visible sur les Figures 2 et 3, a réellement besoin. La désuétude prochaine de l'OTS Manager, couplée au besoin d'un lanceur demeurant pour certains utilisateurs ainsi que la mise en place d'une nouvelle charte graphique pour l'écosystème IndissPlus ont mené au développement de l'IndissPlus Launcher.

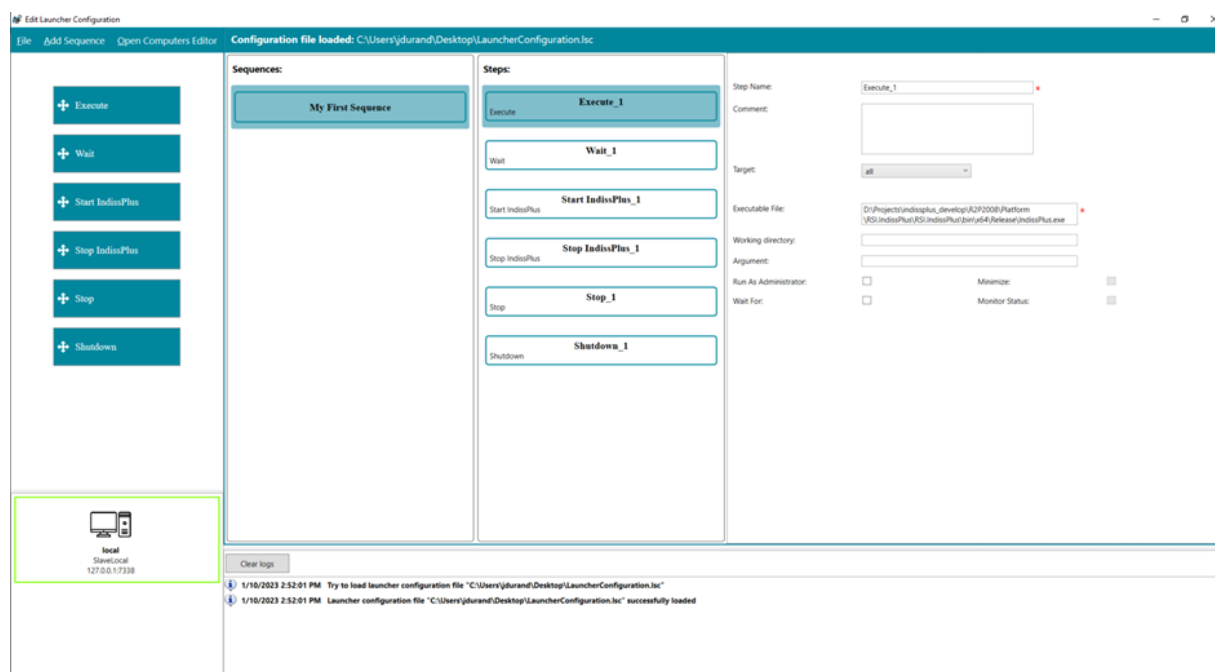


Figure 2 : Interface d'édition de séquence depuis l'OTS Manager

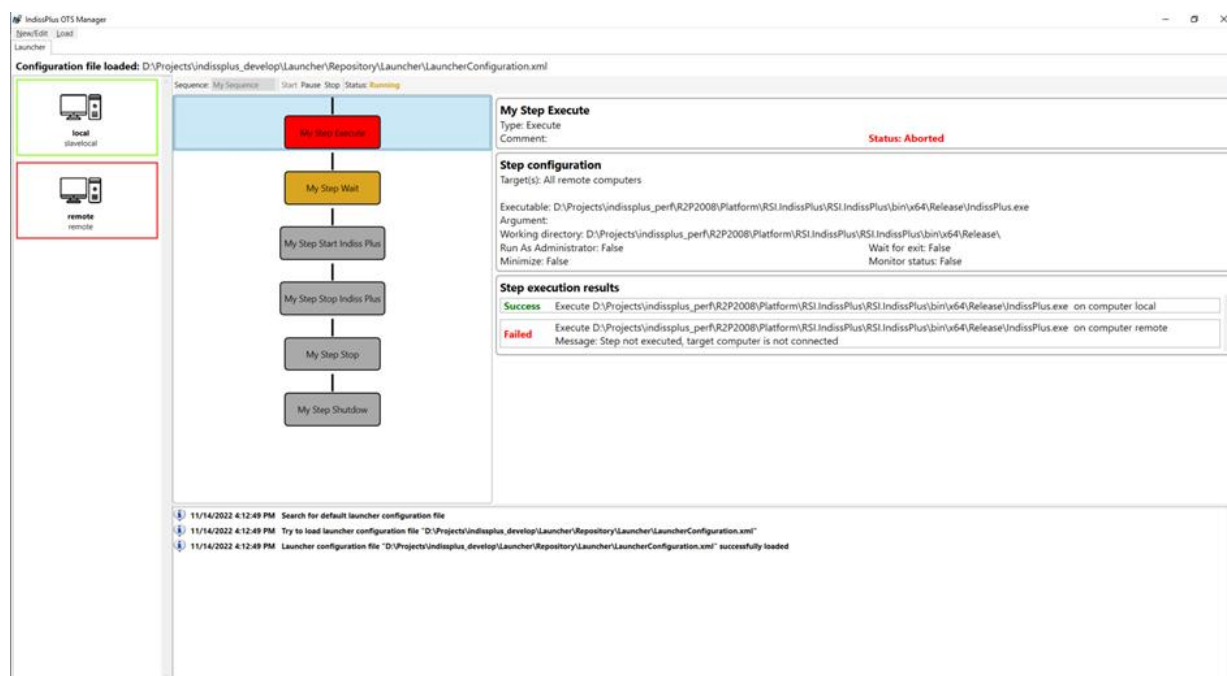


Figure 3 : Interface de suivi d'exécution depuis l'OTS Manager

L'IndissPlus Launcher

Un nouveau développement a donc été engagé depuis quelques années pour produire une version modernisée du mode Launcher de l'OTS Manager. Cette nouvelle version, l'IndissPlus Launcher, adopte les codes graphiques du reste de l'écosystème IndissPlus (cf. Annexe A) et vise à peaufiner les fonctionnalités de l'OTS Manager. La nouvelle application utilisera également à terme le nouveau système de communication utilisé par Compétence Plus, ce système utilise des procédés gRPC* afin de crypter les communications. Quel que soit le système de communication utilisé, le

principe de fonctionnement reste le même, le Launcher ne réalise lui-même aucune des actions programmées par l'instructeur, il se contente, en fonction de la configuration choisie, d'envoyer des commandes aux ordinateurs concernés pour qu'ils exécutent les steps qui leur sont destinés. Cette communication transite aujourd'hui par le LauncherService, le service installé sur chaque poste et chargé de recevoir et d'exécuter les ordres du Launcher, et transitera à terme par le LauncherAgent, déjà utilisé par Compétence Plus. Une fois l'étape exécutée, le poste distant renvoie son résultat au Launcher, qui l'intègre au suivi d'exécution.

À mon arrivée chez Corys, le développement de l'IndissPlus Launcher était en cours depuis plus d'un an. Le projet intégrait déjà les fonctionnalités historiques de Launcher de l'OTS Manager (édition de configurations, de séquences et d'ordinateurs), le lancement de l'exécution des séquences, ainsi que l'interface visuelle commune aux autres applications IndissPlus.

1.3 Objectifs

L'objectif de mon alternance est de finaliser le développement de l'IndissPlus Launcher. Pour ce faire, le travail s'est articulé autour des deux phases principales de l'application. :

Partie édition

Dans cette phase l'utilisateur peut créer une nouvelle configuration, définir des séquences d'actions et y ajouter des ordinateurs. Ces derniers peuvent se voir attribuer un nom et un rôle, permettant d'exécuter une action sur tous les postes partageant le même rôle. Les ordinateurs sont positionnés librement sur un canevas, offrant la possibilité de reproduire la disposition physique de la salle si nécessaire.

Les contributions apportées à cette partie au cours de l'alternance portent principalement sur la correction de bugs techniques et graphiques, ainsi que sur le refactoring* de nombreuses portions de code. Il a également été possible d'implémenter la création de nouvelles configurations, qui n'était pas encore disponible, ainsi qu'une fonctionnalité de validation de séquence permettant à l'utilisateur de vérifier la validité de sa séquence et, à défaut, d'en identifier les erreurs et de les corriger.

Partie exécution

À l'exécution d'une séquence, l'application bascule dans un mode dédié disposant de sa propre interface. L'utilisateur peut y suivre l'avancement de l'exécution, identifier rapidement les défauts de sa séquence, et les corriger, soit en revenant en mode édition, soit en intervenant directement sur le poste opérateur concerné. Il peut également consulter le détail d'une étape en cours, son état, ses éventuelles erreurs ou informations supplémentaires, et ouvrir un panneau de suivi par ordinateur. À l'issue d'une exécution, un rapport contenant toutes les informations des étapes exécutées est généré. L'utilisateur peut choisir de l'exporter s'il souhaite en garder une trace ou le fournir à Corys en cas d'erreur qu'il ne parviendrait pas à corriger.

C'est cette partie qui a constitué l'essentiel du travail d'alternance. Les contributions incluent l'ajout des éléments manquants dans le cycle d'exécution, comme la pause et l'arrêt, ainsi que la conception et le développement complets du mode exécution, réalisés de A à Z. Ce mode permet à l'instructeur de suivre en temps réel l'avancement d'une séquence, de consulter le détail de chaque étape, d'identifier rapidement les erreurs et les ordinateurs concernés, et d'accéder à un rapport d'exécution exportable à l'issue de la séquence. Sa conception a impliqué une phase de

maquettage sur Figma* et l'application de critères ergonomiques, tandis que sa réalisation a nécessité la création de nouvelles structures de données dédiées au stockage des résultats d'exécution.

Restant à faire

À l'approche de la conclusion de mon alternance il reste encore quelques tâches non résolues. Certains bugs persistent encore et toujours, mais seront sans aucun doute résolus dans les semaines à venir. Parmi ces bugs nous retrouvons :

- Une impossibilité de resélectionner un élément graphique, entraînant des problèmes de mise à jour de l'interface (priorité moyenne, estimé à 2 jours/personne).
- Un problème de mise à jour des cibles disponibles en mode édition (priorité haute, estimé à 2 jours/personne).

Une fois le développement abouti, il restera la rédaction de la documentation technique destinée aux utilisateurs, détaillant l'utilisation de l'application, ainsi que l'écriture des fichiers de setup permettant l'installation d'une version officielle du logiciel. Enfin, des tests utilisateurs internes avec les membres de l'équipe de production concernés seront conduits afin d'identifier les éventuelles lacunes des interfaces.

À l'issue de ces tests et des améliorations qui en découleront, l'application sera mise en production, fin septembre, en même temps que Competence Plus. Ce calendrier commun s'explique par le fait que l'IndissPlus Launcher utilise certains services de Competence Plus. Partager la date de mise en production évite de déployer le Launcher une première fois avec des services temporaires voués à disparaître.

1.4 Contraintes et risques

Le projet présente plusieurs contraintes, d'ordre humain, technique et temporel.

Le rythme de l'alternance, et plus précisément les semaines passées à l'IUT, ont naturellement limité le temps disponible en entreprise. La charge importante portée par le développeur référent sur le projet Competence Plus, notamment lors de démonstrations client, a par moments réduit les échanges possibles et nécessité une plus grande autonomie. Ce temps a néanmoins été mis à profit pour traiter les améliorations et corrections réalisables de manière indépendante.

D'un point de vue technique, l'architecture MVVM*, autour de laquelle est construite l'application, impose une rigueur constante et a représenté un investissement initial important, en partie anticipé grâce au mini-projet d'initiation réalisé à l'arrivée. Le mécanisme d'undo/redo* existant s'est également révélé particulièrement fragile, tout refactoring pouvant en affecter le comportement. Ce risque a été mitigé par l'ajout de tests unitaires sur les fonctionnalités critiques. Enfin, la localisation de l'application constitue une contrainte à respecter sur l'ensemble des développements.

Sur le plan temporel, la date de mise en production a été fixée à fin septembre, en parallèle du déploiement de Competence Plus, ce dernier ainsi que le Launcher dépendant de certains services communs.

II. Environnement de travail

II.1 Architecture de l'application

Les applications de l'écosystème IndissPlus partagent de nombreuses fonctionnalités. L'équipe de R&D a donc choisi de généraliser une partie du code de ses applications afin de centraliser les mises à jour et d'éviter la duplication de code. Ce code est hébergé sur le GitLab* de l'entreprise sous le nom de LauncherCommon* et est rendu accessible grâce à des packages NuGet* internes. Les applications peuvent donc importer et utiliser les packages qui leur sont utiles, en Figure 4 sont illustrés tous les packages du LauncherCommon utilisé par l'IndissPlus Launcher. Ainsi les projets LauncherAgent (utilisé par Competence Plus et Competence Plus On-Premise) et LauncherService (utilisé par l'IndissPlus Launcher) utilisent tous deux les packages contenant le code permettant d'exécuter des actions sur des machines distantes. Le package Launcher.Common.Service.ActionsData définit notamment la structure des steps envoyés en exécution ainsi que celle de leurs résultats. Pour permettre à une même étape de cibler un ou plusieurs ordinateurs à la fois, qu'il s'agisse de tous les postes, d'un rôle particulier ou d'un nom précis, les steps concernés héritent de la classe StepWithTarget, qui définit, en plus des propriétés communes à tous les steps, les informations nécessaires à l'identification de ces cibles.

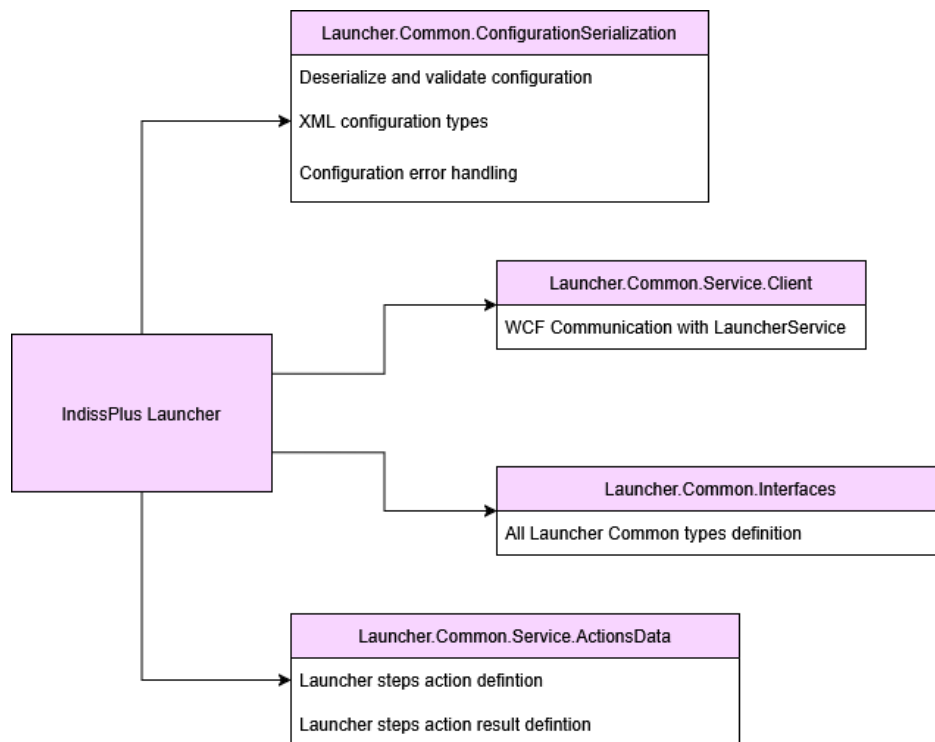


Figure 4 : Schéma des dépendances du projet IndissPlus Launcher, réalisé par M. Durand

Cette mise en commun des modifications a néanmoins des inconvénients puisque chaque mise à jour du Launcher concernant la communication avec les ordinateurs distants a dû nécessiter des actions sur le code du LauncherCommon auquel je n'ai pas accès, ainsi que l'évolution synchronisée des projets Competence Plus.

Pour ce qui est de l'application elle-même, elle est développée en C#, à l'aide du framework .NET Framework 4.8.1, et repose sur une architecture MVVM, le modèle le plus répandu dans les applications de bureau utilisant WPF* (Windows Presentation Foundation) [2][3]. Cette architecture s'organise autour d'une séparation stricte entre trois couches : le Model*, qui contient les données et la logique métier*, la View*, chargée de l'interface graphique et le ViewModel*, qui assure le lien entre les deux. La logique métier désigne l'ensemble des règles de fonctionnement et des traitements de l'application, indépendamment de son interface graphique.

Contrairement à des architectures plus classiques comme le MVC*, le MVVM implique une absence de dépendance directe entre l'interface et la logique métier. Le ViewModel expose les données et les actions disponibles à la View, sans jamais manipuler d'éléments graphiques. La View, quant à elle, lie les données et commandes fournies par le ViewModel aux éléments graphiques rendus disponibles à l'utilisateur. On retrouve les liens entre les différentes couches sur la Figure 5.

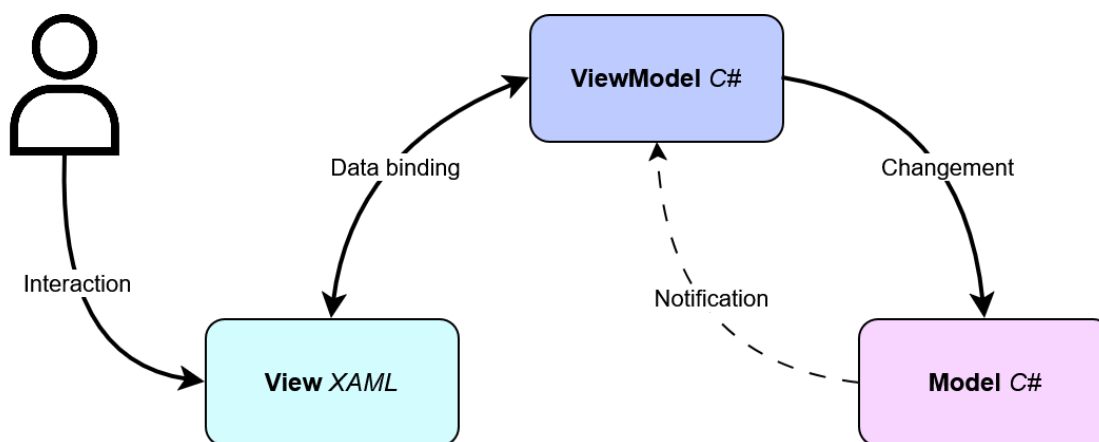


Figure 5 : Schéma explicatif de l'architecture MVVM [4]

Cette architecture permet de simplifier la maintenance du code en séparant la logique métier de l'interface, de pouvoir tester l'application sans interface graphique et de réutiliser la logique métier pour adapter l'application à d'autres plateformes.

L'application expose plusieurs services accessibles depuis toutes les couches de l'application via un ServiceContext*. Ce système permet d'éviter la redondance d'informations et d'en faciliter l'accès sans briser les principes du MVVM. Parmi ceux-ci figurent le LauncherEngineService*, qui pilote l'exécution des séquences, le LocalizationService*, qui gère la traduction de l'interface, ou encore le LauncherConfigurationService*, qui centralise l'accès aux données de la configuration. La sollicitation de ces différents services étant omniprésente dans le développement, il était nécessaire de construire une bonne connaissance du ServiceContext.

II.2 Méthodes de travail

La gestion des projets de l'équipe de R&D est en transition vers un modèle semi-agile. Ce modèle se distingue d'une approche agile complète car il ne suit pas strictement toutes les pratiques Scrum. Certains des outils agiles comme les user stories* et les sprints sont progressivement adoptés mais ne sont pas systématiquement employés. Pour l'IndissPlus Launcher, des user stories ont été rédigées afin de regrouper les fonctionnalités sous un même contexte et de leur donner du sens.

Ces stories suivent le format « As a ..., I want... » (cf. Annexe B), qui permet de recontextualiser chaque demande du point de vue de l'utilisateur. Leur contenu détaille ensuite les critères minimaux à satisfaire pour considérer la story comme complète. Elles ont été rédigées à deux, au fil des réunions hebdomadaires, et classifiées selon leur priorité.

Les réunions hebdomadaires, inspirées des cérémonies agiles, auxquelles j'ai participé ont concerné à la fois les projets Compétence Plus et IndissPlus Launcher. Pour Compétence Plus, ma participation s'explique par l'imbrication des deux projets et par le fait que l'équipe, plus réduite, est constituée pratiquement des mêmes personnes. Ces réunions ont également été l'occasion de présenter les avancées du Launcher et d'obtenir un regard extérieur sur les interfaces et les fonctionnalités en cours de développement afin d'identifier et de corriger les incohérences ergonomiques. Les réunions IndissPlus, plus formelles, rassemblent l'ensemble de l'équipe R&D ainsi que l'équipe de production, sous la conduite du responsable des industries de procédés, qui y fait le point sur l'avancement de chaque projet. Ces réunions permettent de maintenir une vision globale des avancées des différents projets et de réunir les points de vue des deux équipes.

Sur la fin du projet, les user stories restantes ont été organisées en sprints de deux semaines. La répartition du travail s'est faite naturellement selon les accès et compétences de chacun, les développements touchant au code du LauncherCommon, nécessitant une connaissance plus approfondie de l'architecture partagée, ont été pris en charge par le développeur référent, tandis que je me suis chargée de réaliser les fonctionnalités propres au Launcher. Le suivi de ces sprints a été organisé sur le GitLab interne de l'entreprise, via un tableau Kanban représenté ci-dessous sur la Figure 6.

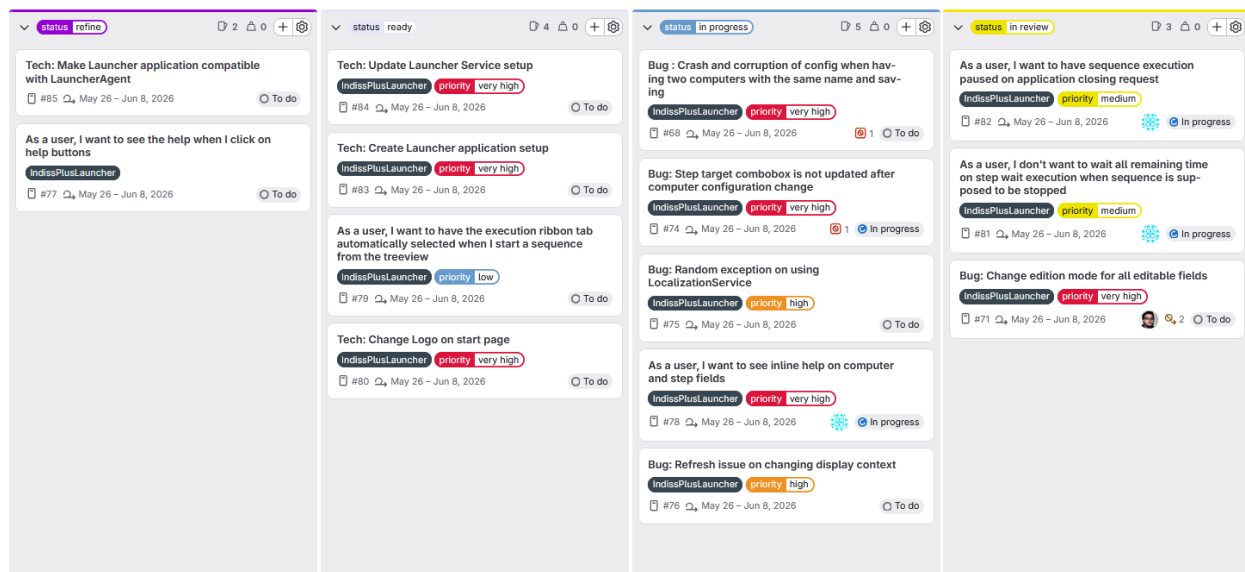


Figure 6 : Tableau Kanban du sprint actuel

Le versionnage du code est effectué grâce à Git* et hébergé sur le Gitlab de l'entreprise, les projets suivent une convention de branches par fonctionnalité, chaque développement étant réalisé sur une branche dédiée, puis intégré à la branche principale du projet. Le code du Launcher est versionné sur une branche de développement distincte du projet principal IndissPlus.

Enfin, vous trouverez ci-dessous, en Figure 7, le planning rétrospectif de mon travail sur ce projet. Ce planning révèle la priorisation des fonctionnalités les plus critiques, ainsi que la superposition



de certaines tâches. En effet la réalisation des différentes maquettes a été effectuée en tâche de fond pour me permettre de prendre un peu de recul sur mon travail de programmation.

Figure 7 : Diagramme de Gantt du planning rétrospectif

III. Refactoring

Le refactoring désigne l'amélioration de la structure interne d'un code sans en modifier le comportement observable. Il permet d'en faciliter la compréhension et la maintenance, et d'identifier les points faibles ou les opportunités d'optimisation. Une part importante du travail d'alternance y a été consacrée, le code existant présentant de nombreuses portions à améliorer : des centaines de lignes commentées, une logique plus complexe que nécessaire et plusieurs principes du Clean Code non respectés [5].

III.1 Application des conseils du Clean Code

Le Clean Code est un ouvrage de Robert C. Martin qui décrit les bonnes pratiques de développement logiciel et a pris une place importante dans la définition de ce qu'est un code de qualité. Sa lecture au cours de l'alternance m'a permise d'en appliquer immédiatement les conseils, tout en conservant un esprit critique. Pratiquer directement les concepts abordés a facilité leur assimilation.

Les apports principaux ont porté sur le choix de noms significatifs pour les variables et fonctions, l'écriture de fonctions courtes et à responsabilité unique, l'élimination des duplications de code selon le principe DRY*, la suppression des commentaires superflus au profit d'un code suffisamment explicite, et la simplification des algorithmes selon le principe KISS*. Une règle pratique a guidé ce travail, dès qu'un commentaire semblait indispensable pour comprendre un bout de code, c'était le signe que ce code méritait d'être refactorisé pour se suffire à lui-même. Face à l'ampleur du refactoring possible, il a également fallu apprendre à se restreindre aux fichiers directement concernés par les modifications en cours, pour éviter de s'éparpiller et de perdre trop de temps.

III.2 StepTargetService (et le pattern observer)

Un refactoring plus conséquent a porté sur la gestion des listes de cibles affichées lors de l'édition des étapes. Ces listes contiennent les noms des ordinateurs et leurs rôles, et doivent être mises à jour lorsque l'un de ces attributs est modifié sur un ordinateur, en tenant également compte du mécanisme d'undo/redo (cf. Annexe C).

La mise en œuvre initiale stockait ces listes, pourtant identiques pour toutes les étapes, dans chaque étape individuellement. Si l'impact mémoire restait négligeable, cette redondance allait à l'encontre du principe DRY et complexifiait inutilement les classes Computer et StepWithTarget, qui se retrouvaient chacune à gérer une responsabilité qui ne leur appartenait pas. En effet, selon le principe de responsabilité unique (SRP*), chaque classe ne devrait avoir qu'une seule raison de changer. Or, la classe Computer se trouvait à la fois responsable de ses propres données et de la mise à jour de listes qui concernaient en réalité toutes les étapes de la séquence. Ne pas respecter ce principe rendait le code plus difficile à faire évoluer. Modifier le comportement des listes de cibles impliquait d'intervenir dans plusieurs classes à la fois, ce qui augmentait le risque d'introduire des bugs et rendait la compréhension du code plus ardue pour un nouveau développeur.

Face à ce problème, la solution initialement retenue consistait à déclencher manuellement une mise à jour de toutes les étapes à chaque modification d'un ordinateur. Cette approche introduisait

cependant un couplage fort entre la classe Computer et les étapes, et s'est révélée particulièrement problématique pour le mécanisme d'undo/redo puisque pour annuler une modification, il aurait fallu conserver non seulement les valeurs précédentes des listes de chaque étape, mais également les valeurs sélectionnées par l'utilisateur pour chacune d'elles, rendant la mécanique d'annulation extrêmement complexe à implémenter de manière fiable.

C'est le pattern Observer* qui a été retenu comme solution. Ce pattern de conception définit une relation entre objets dans laquelle un sujet maintient une liste d'observateurs et les notifie automatiquement de tout changement de son état. Il repose sur deux rôles distincts : le sujet, qui émet les notifications, et les observateurs, qui s'y abonnent et réagissent en conséquence. L'intérêt principal de ce pattern est qu'il permet de découpler le sujet de ses observateurs, le sujet n'a pas besoin de connaître l'identité ou le nombre de ses observateurs, il se contente de signaler qu'un changement a eu lieu [6].

Appliqué au projet, ce pattern se traduit par la création d'un service dédié, le StepTargetService, chargé de centraliser la gestion des listes de cibles. Ce service s'abonne au ConfigurationServiceCallback, qui joue le rôle de sujet et notifie ses abonnés de chaque modification de la configuration, c'est-à-dire lorsque les informations des ordinateurs sont mises à jour. À réception de ces notifications, le StepTargetService met à jour ses listes de cibles, qui sont ensuite transmises aux étapes via la vue. Les valeurs propres à chaque étape continuent quant à elles d'être stockées individuellement.

Ce refactoring permet ainsi de respecter à la fois le principe DRY, en centralisant des données jusque-là dupliquées, et le SRP, en attribuant la responsabilité de la gestion des cibles à un service dédié plutôt qu'aux classes Computer et StepWithTarget. Il rend également le code plus robuste face au mécanisme d'undo/redo, puisque les mises à jour des listes sont désormais gérées à un seul endroit.

III.3 Ajouts des tests unitaires

Les tests sont une partie intégrante du développement logiciel, nous avons bien entendu conduit des tests fonctionnels en parallèle du développement du Launcher, mais également rédigé des tests unitaires. Ces derniers permettent de confirmer le bon fonctionnement de l'application de façon automatique, de découvrir et corriger des bugs si nécessaire et, dans une approche de Clean Code, de figer le comportement actuel afin d'être sûr de ne pas le modifier lors d'un refactoring.

Les tests rédigés portent sur la classe LauncherEngineService, le moteur d'exécution de l'application, chargée de piloter l'exécution de chaque étape en transférant les informations nécessaires au LauncherService. Il s'agit d'une partie centrale du fonctionnement du Launcher, il est donc crucial de s'assurer de son bon fonctionnement, sous peine de rendre l'application inutile et inutilisable.

Ces tests couvrent les trois actions principales du LauncherEngineService : le lancement d'une séquence, la pause et l'arrêt. Ils vérifient au passage à la fois l'état des commandes correspondantes (peut-on lancer, mettre en pause ou arrêter une séquence selon le contexte) et le résultat de l'exécution. Plusieurs scénarios ont été testés pour chaque action : exécution de chacun des six types d'étapes, ciblage d'un ou plusieurs ordinateurs par rôle ou par nom, ou encore comportement de la séquence face à une étape invalide, qui doit alors être interrompue avec le statut

correspondant. Ils sont rédigés selon le format AAA* (Arrange-Act-Assert), qui consiste à séparer clairement la mise en place du contexte de test, le déclenchement de l'action testée, puis la vérification du résultat, afin de faciliter la lecture et la maintenance des tests.

Les tests ont nécessité certaines modifications dans les classes existantes afin de les rendre testables. Tout d'abord, cela a concerné la classe `LauncherEngineService`, puisque les tests avaient besoin de vérifier l'état de la méthode d'exécution sans avoir à utiliser la commande par laquelle passe la vue qui déclenche habituellement l'exécution. La méthode à tester (`ExecuteSequenceMethode`) étant asynchrone et ne possédant aucun point d'ancrage permettant de la rendre accessible depuis l'extérieur, la variable contenant la tâche asynchrone, précédemment créée localement dans la fonction, a été extraite afin de devenir un attribut de la classe, et ainsi pouvoir être récupérée par les tests. Il a également fallu transformer le niveau de protection de la méthode de « `private` » à « `internal` » afin d'en permettre l'accès depuis les tests, sans la rendre publique pour autant et donc maintenir le niveau de sécurité.

Dans un second temps c'est la classe `Computer` qui a dû subir quelques retouches, le `LauncherEngine` faisant directement appel au `DashService` qu'elle contient pour gérer la communication avec les ordinateurs distants. Il fallait donc intercepter ces commandes afin de contrôler l'entièreté de l'environnement de test et de retourner manuellement les codes de succès ou d'échec. Pour ce faire, un constructeur a été ajouté à la classe `Computer` permettant, dans l'environnement de test, d'injecter un `FakeDashService` à la construction d'une instance de `Computer`.

IV. Développement de nouvelles fonctionnalités

IV.1 Ajouts dans le cycle d'exécution de séquence

En parallèle des corrections, plusieurs fonctionnalités essentielles absentes de l'application ont été implémentées : la pause et l'arrêt de l'exécution d'une séquence, la validation des séquences, ainsi que la création de nouvelles configurations puisque jusqu'alors, seule la modification de configurations existantes était possible.

L'élaboration d'une fonction de validation d'une séquence a donné la fonction `ValidateSequence` du `LauncherConfigurationService`. Cette fonction parcourt les étapes de la séquence et récupère les informations sur leur validité depuis la fonction `IsValid`, héritée à travers les différentes sous-classes de la classe `Step`, et permettant à chaque type d'étape de définir ses propres critères de validité de manière cohérente. Lors du parcours des étapes la fonction va également collecter les différents ordinateurs concernés afin de vérifier leur bonne configuration et de tester leur connexion.

L'implémentation de la pause et de l'arrêt a, quant à elle, nécessité de se familiariser avec la programmation concurrente, le programme du BUT s'est révélé fort utile sur ce sujet et m'a permis d'en saisir plus rapidement les mécanismes.

L'exécution successive des étapes est réalisée de manière asynchrone via des `Task*`, de sorte à ne pas figer la vue pendant l'exécution. Cette approche a nécessité l'introduction d'un mécanisme de synchronisation entre `threads*` pour gérer la pause. Celle-ci repose sur un `ManualResetEvent`, nommé `_pauseSequenceEvent`, un mécanisme de synchronisation du framework .NET permettant de bloquer un ou plusieurs threads jusqu'à ce qu'un signal soit émis [7]. Cet objet possède deux états : signalé (set) ou non signalé (reset). Lorsqu'il est non signalé, tout appel à sa méthode `WaitOne()` bloque le thread appelant jusqu'à ce qu'un autre thread appelle `Set()` pour le débloquent.

Dans le cas du Launcher, un appel à `WaitOne()` sur `_pauseSequenceEvent` a été inséré entre l'exécution de chaque étape de la séquence. Ainsi, lorsque l'instructeur déclenche une pause, l'événement est réinitialisé (`Reset()`), ce qui suspend l'exécution dès la fin de l'étape en cours, sans interrompre celle-ci en pleine exécution. À la reprise, l'appel à `Set()` débloquent le thread en attente et l'exécution de la séquence se poursuit normalement à l'étape suivante.

La commande Stop a notamment posé des difficultés spécifiques liées à la gestion des threads puisqu'une inversion des deux lignes de code présentées ci-dessous avait suffi à provoquer des comportements d'échappement des threads.

```
private void StopExecution()
{
    _cancellationTokenSource.Cancel(); -> Envoie la demande d'arrêt
    _pauseSequenceEvent.Set(); -> Annule la potentielle pause de l'exécution
}
```

Dans cette méthode l'ordre de ces deux lignes est important puisque qu'il faut impérativement envoyer la notification de la demande d'arrêt aux threads avant de débloquent la pause de l'exécution. Il m'avait tout d'abord paru logique de commencer par débloquent la pause, ce qui

conduisait au lancement d'instructions supplémentaires puisque la notification de stop n'arrivait pas à temps.

IV.2 Ajout d'un mode exécution

Le principal apport fonctionnel de l'IndissPlus Launcher par rapport à l'OTS Manager est l'ajout d'un suivi de l'exécution. Ce suivi permet de suivre en temps réel l'exécution d'une séquence, d'identifier les erreurs au fil des étapes et de conserver temporairement un rapport d'exécution.

Afin de mieux comprendre les besoins de l'utilisateur final j'ai réalisé une persona (Figure 8) à partir des conversations que j'ai pu avoir avec les tuteurs quant aux besoins de l'application.

	Storytelling :	Besoins :
	Je m'appelle Nathalie j'ai 47 ans et je suis instructrice chez ... depuis maintenant 8 ans. J'ai déjà utilisé l'OTS Manager pour mettre en place les sessions de formation.	J'ai besoin de pouvoir lancer la mise en place automatique des postes de la salle d'examen. J'ai besoin d'identifier rapidement quelles sont les erreurs et sur quels postes elles sont survenues.
Nom :	Intitulé de poste :	Objectifs :
Nathalie	Instructrice de sessions de formation, Ingénieure en chimie industrielle	❖ Identification des erreurs ❖ Diagnostic rapide du problème et de la solution ❖ Identification des postes informatiques concernés
Âge :		
47 ans		

Figure 8 : Persona d'un instructeur type

Les besoins fonctionnels identifiés sont l'accès aux informations détaillées sur l'exécution des étapes, la remontée rapide des informations en cas d'erreurs et la communication de l'avancement de l'exécution en temps réel.

Les besoins non-fonctionnels identifiés sont la bonne visibilité des erreurs, la compréhension du passage entre mode édition et exécution. Afin de répondre au besoin de visibilité, nous avons choisi de suivre certains des critères de Bastien et Scapin [8].

Le critère de guidage se traduit par un regroupement des éléments relatifs aux étapes d'un côté et des éléments relatifs aux ordinateurs de l'autre, ainsi que par une amélioration de la lisibilité des couleurs. L'application a également été testée avec différents filtres de daltonisme afin de s'assurer qu'elle reste lisible pour tous les utilisateurs. Le critère de densité informationnelle a orienté le choix de rendre les nombreux détails d'exécution disponibles à la demande plutôt que systématiquement affichés. Le critère d'homogénéité a guidé la conception du mode exécution, qui reprend le fonctionnement du mode édition en adaptant uniquement le résultat des actions : un clic sur une étape, par exemple, affiche les détails de son exécution plutôt que ses propriétés. Enfin, le critère de signifiante des codes et dénominations a conduit à représenter certaines données par des couleurs aux significations reconnues, comme l'état des étapes, et par des symboles déjà

utilisés dans les logs et dans les autres applications de l'écosystème IndissPlus. Cela participe d'une meilleure intuitivité et par conséquent d'un gain de temps.

L'ajout du mode exécution a également impliqué la création d'un bouton pour en sortir. Conformément à la convention retenue dans l'application, selon laquelle toute action doit être accessible depuis au moins deux points de l'interface, deux boutons ont été ajoutés : l'un dans le menu ruban « Exécution », l'autre en haut à droite de la fenêtre, accessible à tout moment quelle que soit la vue affichée (cf. Annexe D).

IV.3 Mode exécution : Séquences

Conception

Une première façon de suivre l'exécution en temps réel de la séquence est de consulter la fenêtre des séquences, où les étapes se colorent en fonction de leur statut. Cette fonctionnalité était déjà en place à mon arrivée, le travail a donc consisté à en améliorer la lisibilité en éclaircissant les couleurs, conformément au critère de lisibilité de Bastien et Scapin*.

Pour la conception du suivi d'exécution du point de vue des étapes, une première piste a été d'afficher les résultats dans un panneau latéral mis à jour en temps réel, à la manière d'un rapport d'exécution. Il existait déjà une forme rudimentaire de ce suivi dans ce même panneau (cf. Annexe E), ainsi le travail a consisté à en soigner le rendu et à y ajouter les informations essentielles manquantes, notamment celles relatives aux ordinateurs. Ci-dessous en Figure 9, la maquette réalisée sur Figma, les informations retenues sont l'état de l'étape, le temps restant s'il s'agit d'une étape Wait, s'il s'agit à l'inverse d'une étape relative à des ordinateurs, le résultat de l'exécution de cette étape sur chacun des ordinateurs concernés, ainsi les informations complémentaires disponibles et un éventuel message d'erreur.

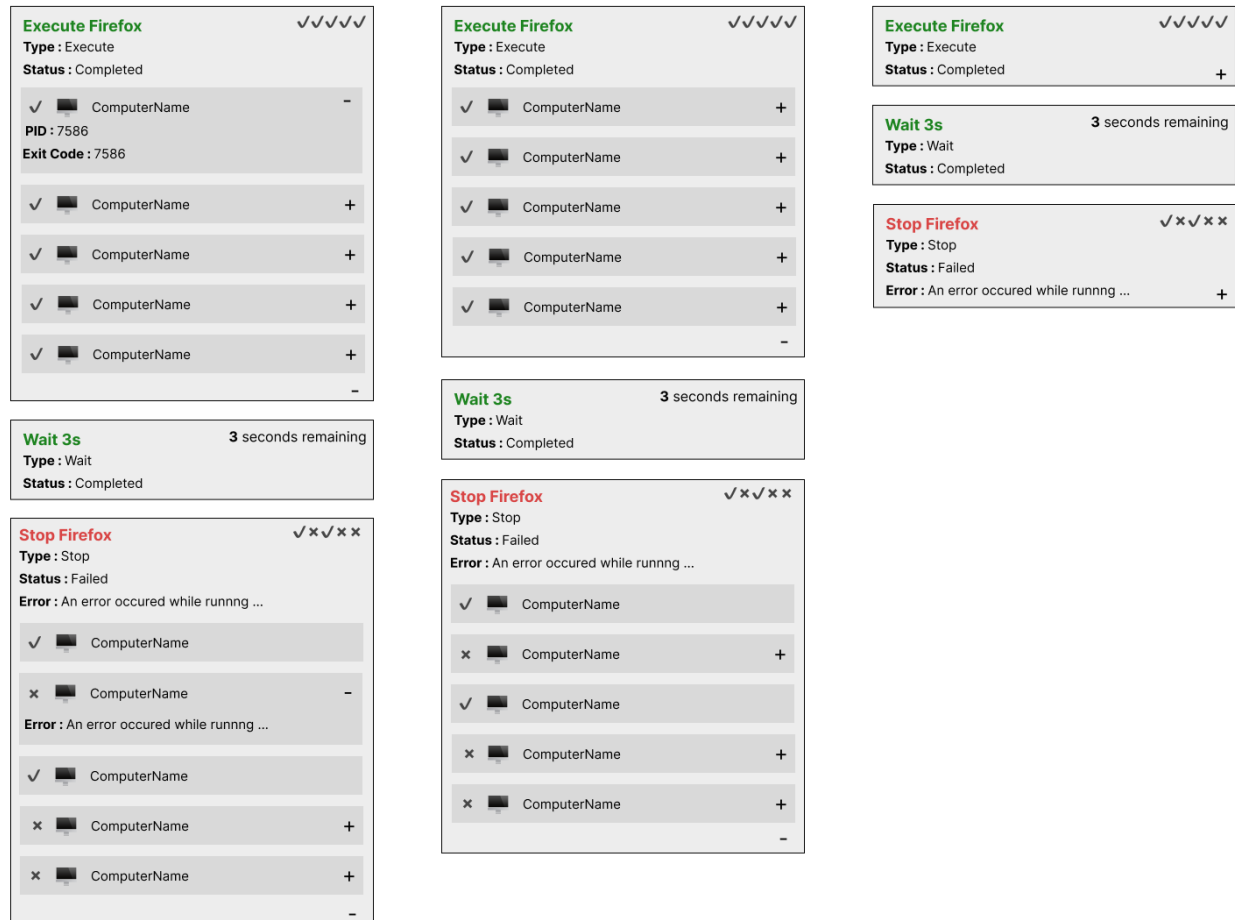


Figure 9 : Maquettes des rapports d'exécutions

Ce format a néanmoins été écarté, et réutilisé pour les rapports d'exécution, car il risquait de ne pas permettre à l'utilisateur de faire facilement le lien entre un résultat et l'étape correspondante. Plusieurs idées ont d'abord été esquissées sur papier, puis deux propositions ont été formalisées sous forme de maquettes sur Figma. La première (Figure 10) propose d'afficher les résultats des étapes directement dans la fenêtre de la séquence, la seconde (Figure 11) propose de n'afficher les détails dans le panneau latéral qu'après un clic de l'utilisateur sur l'étape concernée.

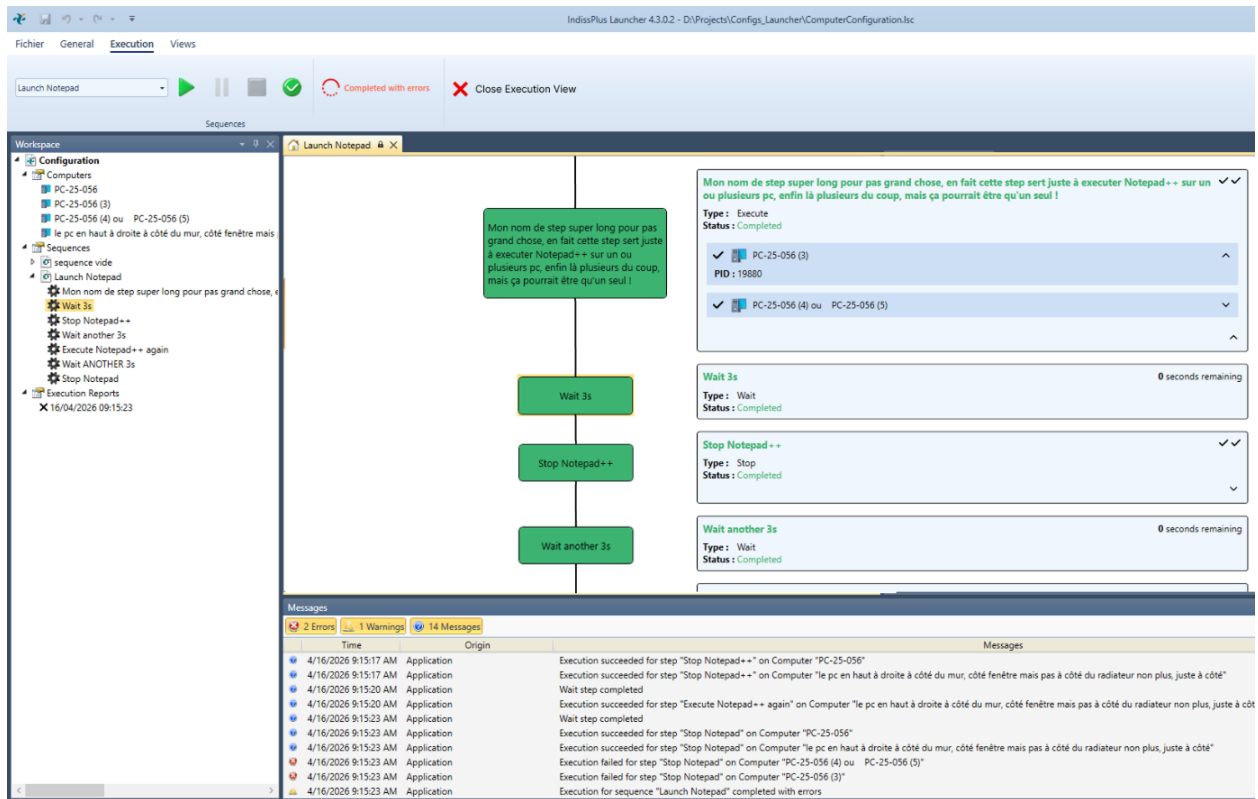


Figure 10 : Maquette de suivi d'exécution des étapes avec affichage dans la fenêtre de séquence

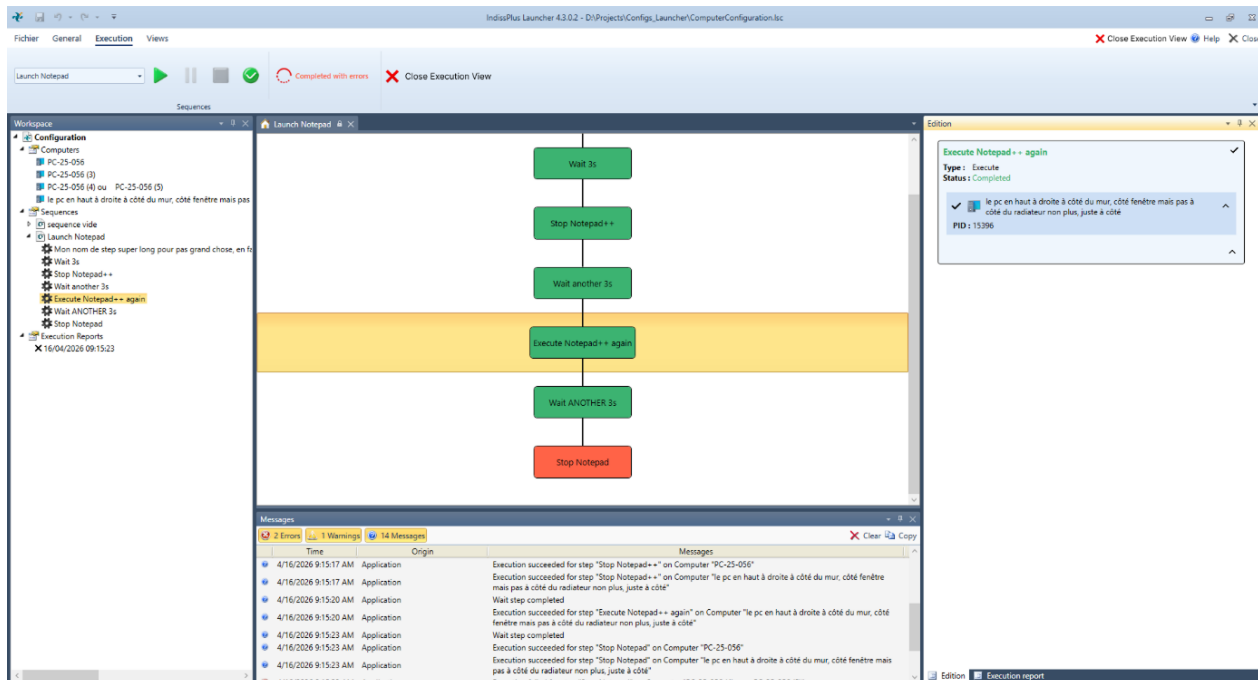


Figure 11 : Maquette de suivi d'exécution des étapes avec affichage dans le panneau latéral

C'est la seconde option qui a été retenue par le chef de projet, d'abord par cohérence avec le comportement attendu par un utilisateur habitué au mode édition (où le clic sur une étape en affiche déjà les détails), puis par respect du critère ergonomique de densité d'information.

Réalisation

Pour implémenter ce suivi, de nouvelles classes `StepResult` ont été créées afin de stocker les informations liées à la dernière exécution de chaque étape. Le changement entre rapport d'exécution et étape par étape a tout de même apporté quelques modifications puisque les résultats étaient initialement complètement indépendant des étapes, ce qui induisait de la redondance dans les informations et introduisait de potentiels erreurs de mise à jour des données. On retrouve dans le diagramme de classe final présenté en Figure 12 les détails de la classe `StepResult`. Elle regroupe les attributs communs à tous les résultats. Par héritage, suivant le même principe que celui des classes `Step`, des attributs plus spécifiques viennent s'y ajouter selon le type d'étape concernée. La sous-classe `StepWithTargetResult`, contient également une nouvelle classe `ComputerStepResult`, qui regroupe les résultats propres à un ordinateur : un booléen indiquant le résultat de son exécution ainsi que les informations associées, comme le PID de l'application qu'il a pu lancer.

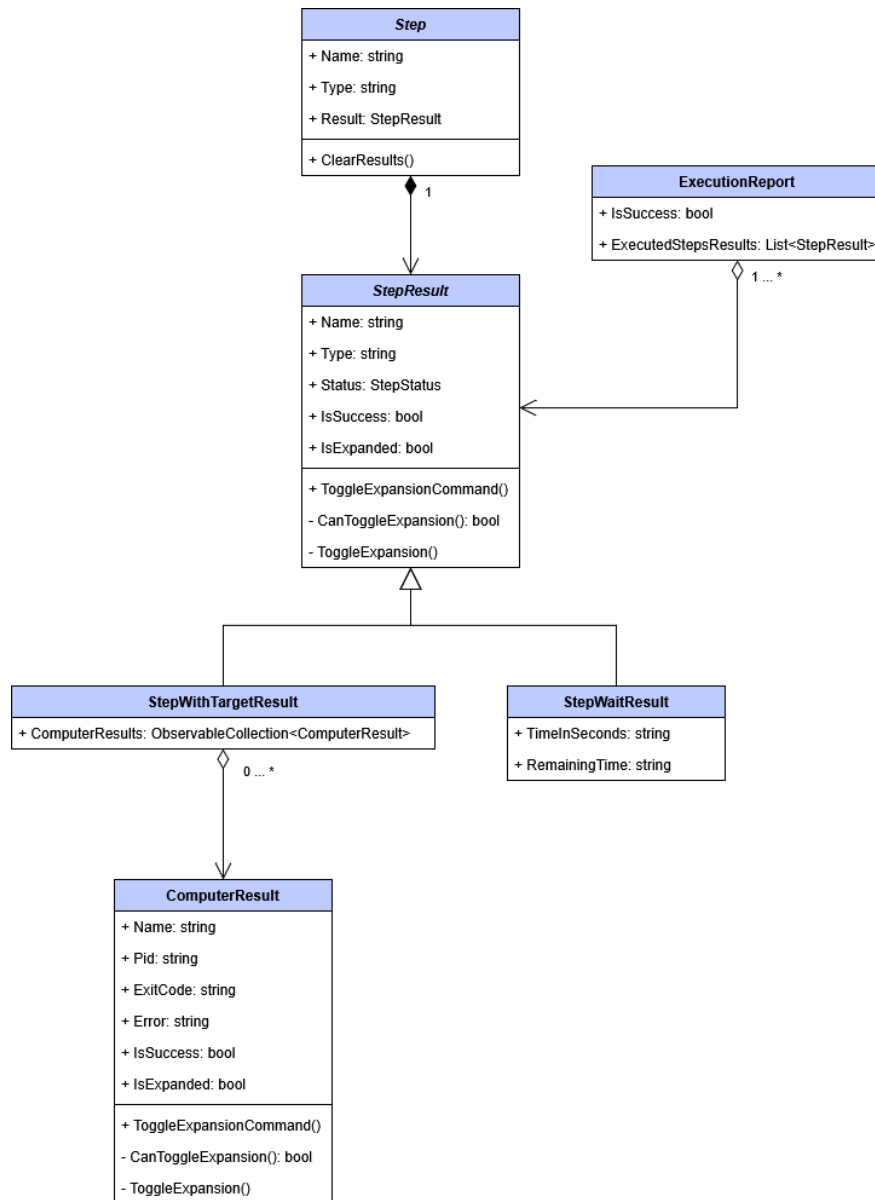


Figure 12: Diagramme des classes *StepResult* et *ComputerResult*

Comme mentionné dans la conception des maquettes, le suivi d'exécution des étapes a d'abord été réalisé comme un rapport d'exécution en temps réel. Pour la réalisation finale du suivi, étape par

étape dans le panneau latéral des détails, j'ai donc pu récupérer le code développé au préalable pour les rapports d'exécution.

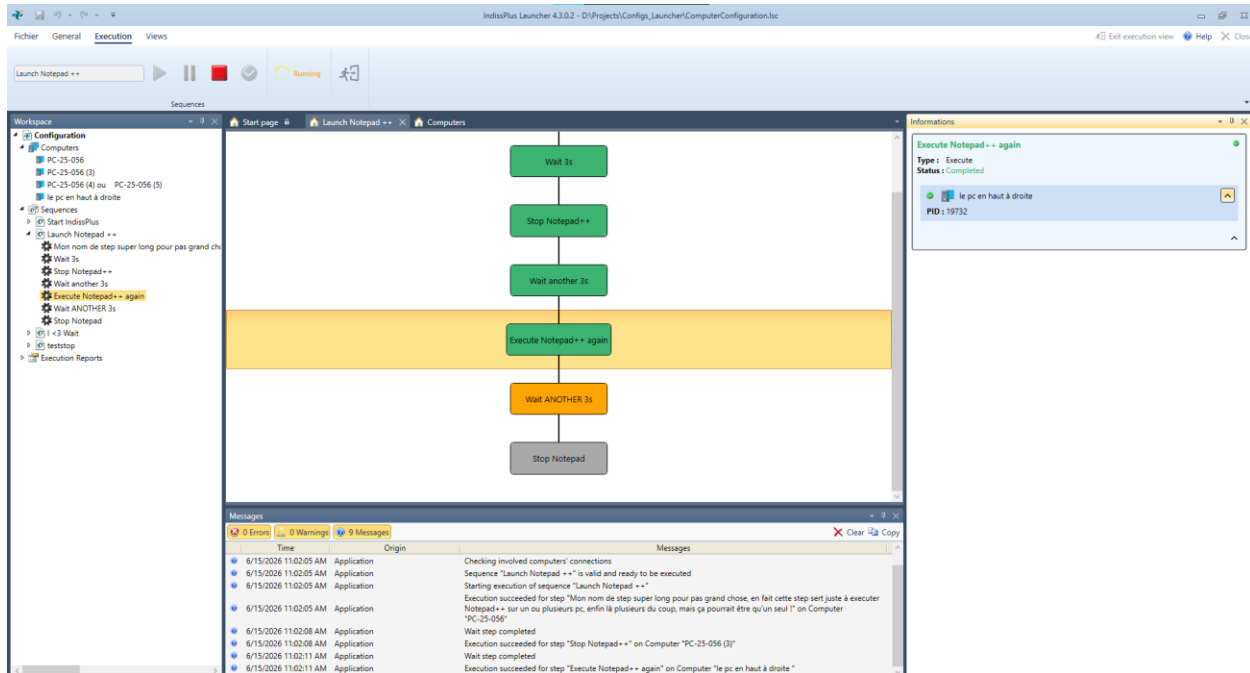


Figure 13: Résultat du suivi d'exécution depuis le point de vue des étapes

IV.4 Mode exécution : Ordinateurs

Conception

Il était également souhaité que l'exécution puisse être observée depuis la fenêtre des ordinateurs, pour poursuivre l'idée de placer les ordinateurs sur le canevas en imitant la salle informatique physique de l'instructeur et ainsi qu'il repère facilement quel ordinateur se trouve en erreur. Cette vue répond à un besoin complémentaire à celui de la vue par séquence, là où cette dernière offre une vision globale de l'avancement de l'exécution étape par étape, la vue ordinateurs permet à l'instructeur de se concentrer sur un poste spécifique et d'en consulter l'historique d'exécution. Ceci s'avère particulièrement utile lorsqu'une erreur est remontée sur un ordinateur et qu'il faut en saisir rapidement le contexte.

Comme pour les détails des étapes, l'implémentation de cette vue avait déjà été engagée (cf. Annexe F). Une phase de conception sur papier a précédé la réalisation des maquettes définitives sur Figma, visible en Figure 14. Pour ces maquettes, j'ai choisi de reprendre l'affichage en liste chaînée de la fenêtre d'une séquence afin de pouvoir suivre l'avancement de l'exécution du point de vue d'un ordinateur comme on le ferait pour une séquence. J'affiche donc les étapes relatives à l'ordinateur observé, ainsi que les résultats le concernant.



Figure 14 : Maquettes de suivi d'exécution depuis la fenêtre des ordinateurs

Réalisation

Les étapes affichées ne concernant qu'un seul ordinateur, je n'ai pas pu reprendre la classe `StepResult` telle quelle puisqu'elle contenait trop d'informations qui ne m'étaient pas pertinentes dans le cas présent. J'ai donc décidé de créer une autre classe pour ne stocker que les informations utiles, la classe `StepResultPerComputer`, décrite dans le diagramme ci-dessous, en Figure 15.

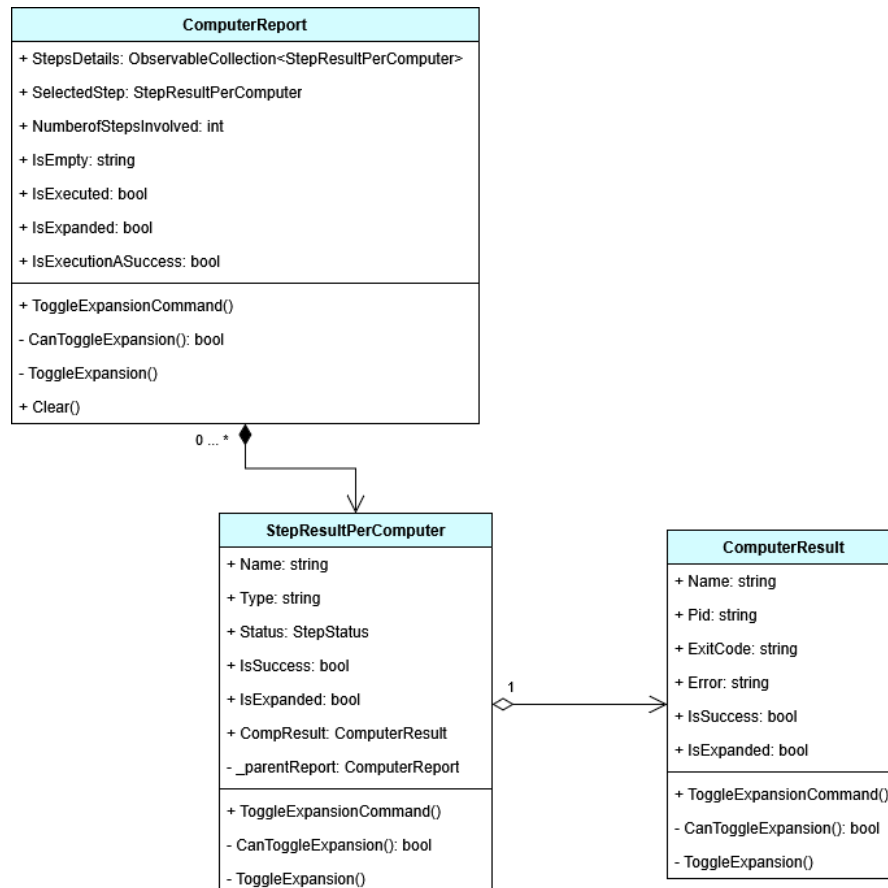


Figure 15 : Diagramme de la classe `StepResultPerComputer`

Cette nouvelle classe est uniquement à destination de la vue et ne sera pas sérialisée ou exportée dans les rapports d'exécution comme le seront les sous-classes de `StepResult`.

L'implémentation concrète de la vue s'est effectuée sans difficulté notable, ci-dessous en Figure 16, le résultat final.

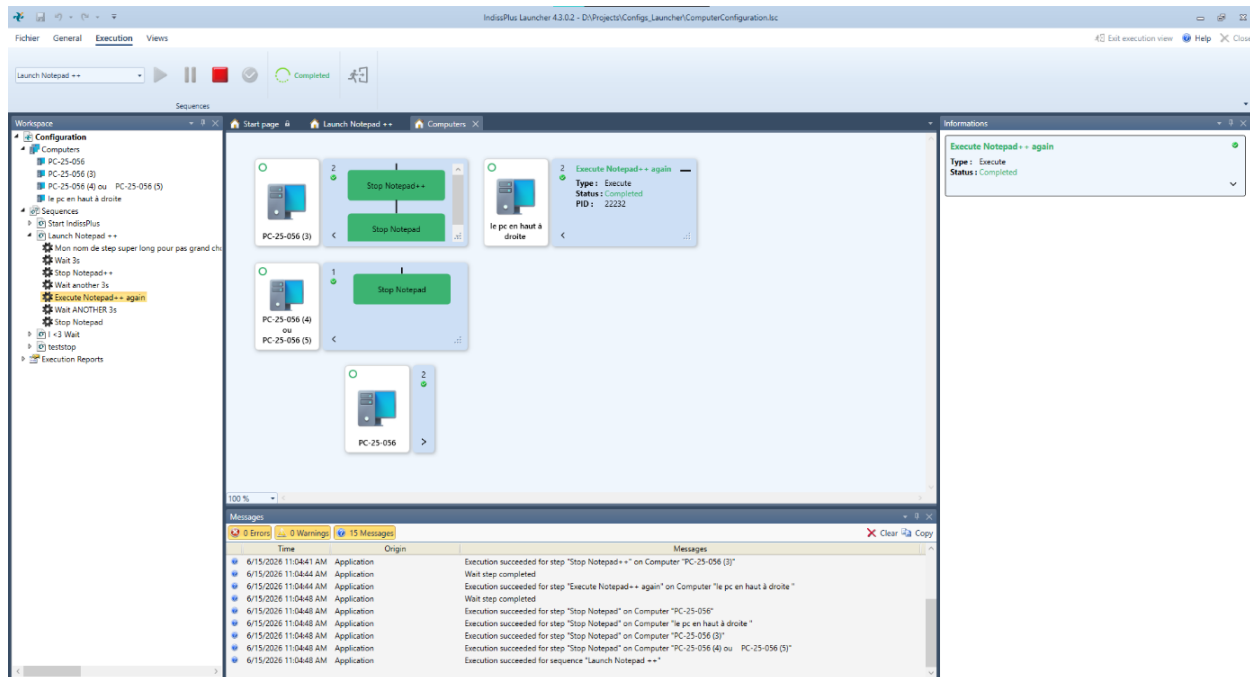


Figure 16 : Résultat du suivi d'exécution depuis le point de vue des ordinateurs

V. Conclusion

Ainsi, l'IndissPlus Launcher, qui à mon arrivée s'avérait déjà bien avancé, a considérablement évolué au cours de cette alternance. Les fonctionnalités manquantes du cycle d'exécution ont été implémentées, le code existant a été restructuré et amélioré, et le mode exécution, qui constitue le cœur du travail réalisé, a été conçu et développé dans sa quasi-totalité. Quelques bugs persistent et seront traités dans les semaines à venir, mais le projet est en bonne voie pour une mise en production prévue fin septembre, en parallèle de Competence Plus. Ce travail technique, mené sur la quasi-totalité de mon année d'alternance, ne représente toutefois qu'une partie de ce que cette expérience m'a apporté.

Sur le plan professionnel, cette alternance m'a beaucoup apporté. D'un côté, j'ai pu prendre en main un projet de grande envergure, avec tout ce que cela implique : comprendre une architecture existante, identifier les conventions adoptées par l'équipe, et améliorer le code existant sans en déstabiliser les fondations. De l'autre, j'ai pu découvrir et monter en compétence sur des technologies nouvelles, le C#, le WPF et l'architecture MVVM, dont la maîtrise progressive m'a permis d'aborder des développements de plus en plus complexes avec de plus en plus d'autonomie.

Sur le plan personnel, cette alternance a été l'occasion de découvrir la littérature technique propre au domaine du développement logiciel. La lecture du Clean Code, appliquée directement au projet, a renforcé mon intérêt pour la qualité de code et pour les questions de maintenabilité et de lisibilité, qui dépassent largement le simple fait que le programme fonctionne. Cette curiosité nouvelle m'a donné envie de continuer à explorer d'autres ouvrages sur le sujet, au-delà du cadre de cette alternance, et a contribué à révéler mon souhait de continuer à approfondir ce domaine dans mon parcours personnel et professionnel. Cette année fut enfin l'occasion d'une réflexion plus large sur les responsabilités environnementales et sociétales de l'entreprise qui m'a accueillie.

Corys est très impliquée dans ses responsabilités écologiques. L'entreprise est notamment certifiée ISO 9001, norme qui garantit la qualité et la pérennité des produits développés, ainsi qu'ISO 14001, qui vise à structurer et à améliorer sa performance environnementale tout en contribuant aux Objectifs de Développement Durable. La clientèle de l'entreprise se peuple de plus en plus d'entreprises développant des énergies durables, même si historiquement la plus grande partie d'entre elles reste tournée vers les énergies fossiles, comme la pétrochimie. Cette base historique de clients du pétrole fut au cœur de nombreuses discussions avec des collègues engagés dans des démarches écologiques personnelles. Ces échanges constructifs ont fait émerger l'idée que, si ces procédés restent dommageables pour la cause environnementale, notre travail consiste avant tout à renforcer la sécurité de processus qui existeraient de toute façon, avec ou sans notre participation. Cette réflexion s'applique d'ailleurs directement à l'IndissPlus Launcher puisqu'en facilitant la mise en place des sessions de formation sur simulateur, l'application contribue à mieux préparer les opérateurs aux situations à risque sans qu'ils aient à intervenir sur les installations réelles, ce qui limite les conséquences d'une éventuelle erreur humaine. Ces discussions m'ont surtout permis de prendre du recul sur mon propre positionnement face à ces enjeux, et de mieux percevoir la valeur, même indirecte, qu'un travail technique peut avoir sur le plan environnemental.

Enfin, sur le plan sociétal, l'entreprise entreprend de nombreux projets pour rassembler ses collaborateurs et veiller à la bonne intégration de chacun. Le CSE organise ainsi de nombreuses activités, comme des voyages, des soirées, des dégustations d'œnologie, des jeux de société, ou

encore un concours de photographie. L'entreprise soutient enfin de nombreuses causes sociales, notamment via des collectes régulières. Une attention particulière est également portée à l'inclusion des personnes en situation de handicap et à la prévention des risques psychosociaux, avec des référents dédiés disponibles pour accompagner les salariés qui en auraient besoins en faveur d'associations locales.

Ainsi se conclut (presque) mon temps en tant que développeuse chez Corys. Cette année à leurs côtés m'a permise de consolider mon souhait de placer la qualité de code au cœur de mon travail, et de continuer à en apprendre toujours plus sur les façons de produire un code propre et maintenable. Elle aura également été l'occasion d'avoir des échanges constructifs sur l'avenir du métier de développeur, si certains sont plus optimistes que d'autres sur la place que prendra l'IA dans le métier, tous s'accordent sur la nécessité, à plus ou moins long terme, de l'intégrer à notre travail, voire d'en devenir les superviseurs, ce qui est un métier qui ne m'intéresse pas le moins du monde. C'est donc avec cette conviction que je poursuis mon parcours académique en master l'an prochain, dans l'espoir de pouvoir, par la suite, exercer aussi longtemps que possible le métier qui me plaît.

Glossaire

AAA (Arrange-Act-Assert) : Méthode d'organisation des tests unitaires consistant à séparer la mise en place du contexte de test, le déclenchement de l'action testée, puis la vérification du résultat.

Bastien et Scapin (critères de) : Ensemble de critères ergonomiques destinés à évaluer et améliorer l'utilisabilité des interfaces logicielles.

BPMN (Business Process Model and Notation) : Notation graphique standardisée permettant de modéliser les étapes et les flux d'un processus métier.

C# : Langage de programmation orienté objet développé par Microsoft, principalement utilisé avec le Framework .NET pour le développement d'applications.

Compétence Plus : Application développée par Corys permettant d'organiser des sessions de formation en ligne ou sur site (Compétence Plus On-Premise).

DRY (Don't Repeat Yourself) : Principe de développement logiciel visant à éviter la duplication de code en extrayant les portions répétées dans des fonctions ou des composants réutilisables.

Figma : Application web de conception d'interfaces utilisée pour réaliser des maquettes graphiques.

Git : Système de gestion de versions permettant de suivre l'évolution du code source et de travailler de manière collaborative.

GitLab : Plateforme de gestion de dépôts Git intégrant des outils de collaboration, de suivi et d'intégration continue.

gRPC : Protocole de communication permettant l'échange de données entre applications de manière chiffrée et performante.

Industries de procédés : Ensemble des secteurs industriels impliquant la transformation de matières premières en produits finis par des procédés chimiques ou physiques, regroupant notamment le pétrole, le raffinage, l'hydrogène et la pétrochimie.

KISS (Keep It Simple, Stupid) : Principe de développement logiciel préconisant de privilégier les solutions les plus simples possibles.

Launcher : Application permettant d'automatiser le lancement et la gestion d'autres applications sur des machines locales ou distantes.

LauncherCommon : Package interne à Corys regroupant les services de communication partagés entre l'IndissPlus Launcher et Compétence Plus.

LauncherConfigurationService : Service centralisé de l'IndissPlus Launcher permettant d'accéder aux données de la configuration en cours.

LauncherEngineService : Service de l'IndissPlus Launcher chargé de piloter l'exécution des séquences.

LauncherService : Service installé sur chaque poste distant, chargé de recevoir et d'exécuter les ordres transmis par le Launcher.

LocalizationService : Service gérant la traduction de l'interface de l'application dans la langue de l'utilisateur.

Logique métier : Ensemble des règles de fonctionnement et des traitements propres à une application, indépendamment de son interface graphique.

Model : Composant de l'architecture MVVM contenant les données et la logique métier de l'application.

MVC (Model-View-Controller) : Architecture logicielle séparant les données, l'interface et la logique de contrôle.

MVVM (Model-View-ViewModel) : Architecture logicielle séparant les données, la logique métier et l'interface graphique afin de faciliter la maintenance et les tests.

NuGet : Gestionnaire de paquets permettant de distribuer et d'intégrer des bibliothèques de code dans les projets .NET.

Observer (pattern) : Patron de conception permettant à un objet de notifier automatiquement d'autres objets abonnés lorsque son état change.

OTS Manager : Application historique de Corys permettant d'automatiser le lancement et l'arrêt des applications lors des sessions de formation, dont l'IndissPlus Launcher est le successeur modernisé.

Refactoring : Processus consistant à améliorer la structure interne d'un code sans en modifier le comportement observable, dans le but d'en faciliter la maintenance et la lisibilité.

ServiceContext : Mécanisme de l'IndissPlus Launcher permettant d'exposer les services à l'ensemble des couches de l'application.

SRP (Single Responsibility Principle) : Principe de conception logicielle stipulant que chaque classe ou fonction ne doit avoir qu'une seule responsabilité.

Step : Étape d'une séquence dans l'IndissPlus Launcher, représentant une action à exécuter sur un ou plusieurs ordinateurs.

Task : Unité de travail asynchrone en C#, permettant l'exécution d'actions en parallèle du fil d'exécution principal de l'application.

Thread : Unité d'exécution permettant à un programme d'effectuer plusieurs tâches en parallèle.

Undo/Redo : Mécanisme permettant à l'utilisateur d'annuler ou de rétablir ses dernières actions dans une application.

User story : Description concise d'une fonctionnalité du point de vue de l'utilisateur, suivant généralement le format « As a..., I want... ».

View : Composant de l'architecture MVVM correspondant à l'interface graphique de l'application.

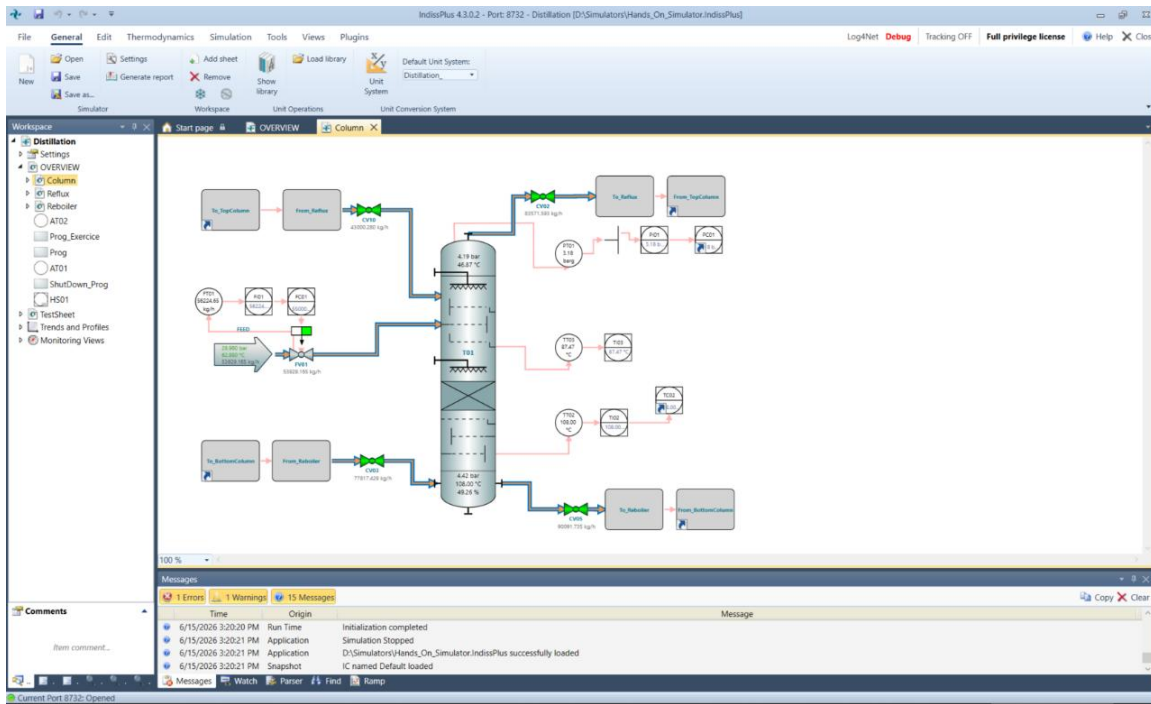
ViewModel : Composant de l'architecture MVVM assurant le lien entre le Model et la View, en exposant les données et les actions disponibles à l'interface.

WPF (Windows Presentation Foundation) : Technologie .NET permettant de créer des interfaces graphiques riches pour les applications Windows.

Bibliographie

- [1] Corys. Corys Dynamic Simulation [en ligne] Disponible sur : <https://www.corys.com/en/> (consulté le 16/12/2025)
- [2] Microsoft. Modèle-View-View Modèle (MVVM) [en ligne] Disponible sur : <https://learn.microsoft.com/fr-fr/dotnet/architecture/maui/mvvm> (consulté le 13/06/2026)
- [3] Microsoft. Vue d'ensemble de Windows Presentation Foundation [en ligne] Disponible sur : <https://learn.microsoft.com/fr-fr/dotnet/desktop/wpf/overview/> (consulté le 13/06/2026)
- [4] Etienne Monier. Architecturer ses applications JS à l'aide du pattern MVVM [en ligne] Disponible sur : <https://www.docdoku.com/blog/2015/02/17/architecturer-ses-applications-js-pattern-mvvm/> (consulté le 18/06/2026)
- [5] Martin R.C. Clean Code: A Handbook of Agile Software Craftsmanship, 2e édition. Addison-Wesley Professional, octobre 2025.
- [6] Shvets A. Refactoring Guru, Design Patterns [en ligne] Disponible sur : <https://refactoring.guru/design-patterns> (consulté le 11/06/2026)
- [7] Microsoft. ManualResetEvent Class [en ligne] Disponible sur : <https://learn.microsoft.com/en-us/dotnet/api/system.threading.manualresetevent?view=netframework-4.8.1> (consulté le 20/06/2026)
- [8] Usabilis. Les critères ergonomiques de Bastien et Scapin [en ligne] Disponible sur : <https://www.usabilis.com/criteres-ergonomiques-bastien-et-scapin/> (consulté le 11/06/2026)

Annexe A : Capture d'écran d'IndissPlus



Annexe B : User-story type

As a user, I want to export my execution report

Issue created 4 months ago by jdurand

Prerequisites:

- A Launcher configuration is open in the application
- A sequence tab is open
- A sequence to execute is selected
- User start and finish the execution of the sequence

Description

Once the execution of the sequence is done and the report is generated, the user should be able to export it to multiple formats.

Acceptance Criteria

List the conditions that must be met for this user story to be considered complete.

- ☒ On execution details panel, display a button that generate a file containing every information from the execution report
- ☒ On execution details panel, display something that allow the user to choose the desired format
- ☒ Allow user to choose where to save the execution report
- ☒ Save report to desired format
 - ☒ Json
 - ☒ Xml

✓ 6 of 6 checklist items completed · Edited 3 months ago by lgoy

Status

Done

Assignees

lgoy

Labels

IndissPlusLauncher x priority low x

Parent

None

Weight

None

Milestone

None

Iteration

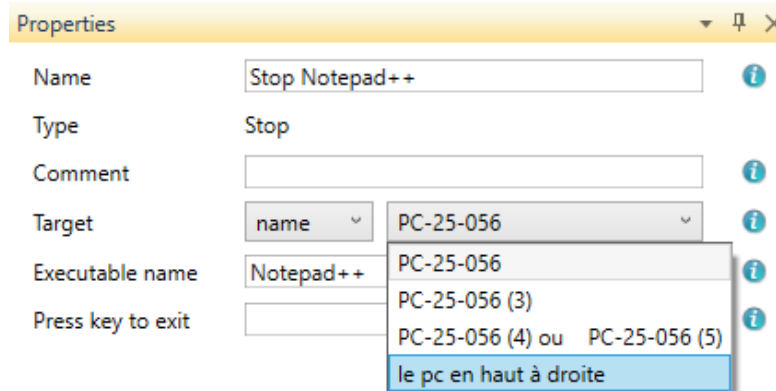
None

Dates

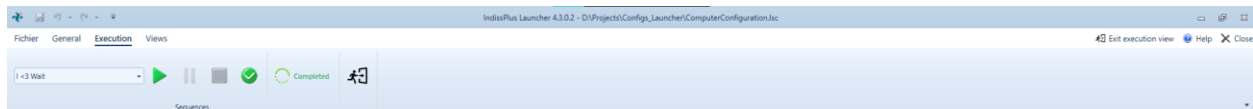
Start: None

Due: None

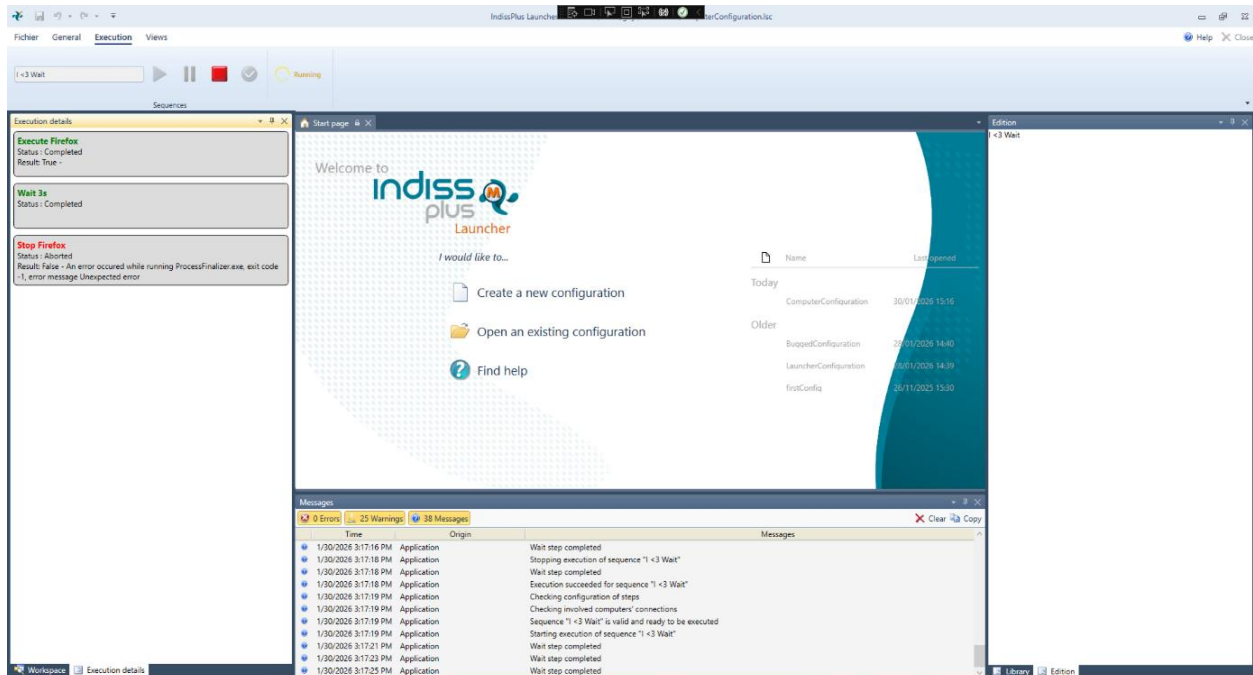
Annexe C : Liste de cibles disponibles pour une étape



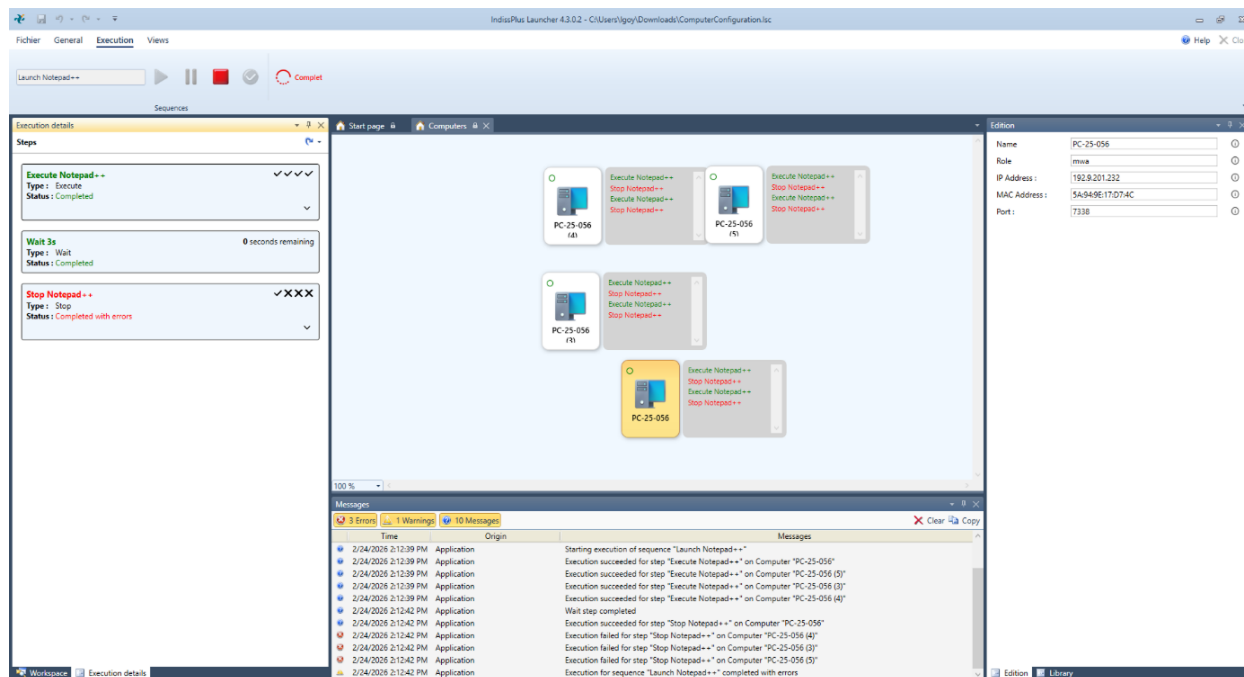
Annexe D : Bandeau d'exécution de l'IndissPlus Launcher



Annexe E : Détails d'exécution des étapes à mon arrivée



Annexe F : Affichage des détails d'exécution des ordinateurs à mon arrivée



Résumé :

Ce rapport présente le travail réalisé dans le cadre d'une alternance au sein de la société Corys, entreprise grenobloise spécialisée dans le développement de simulateurs industriels. La mission principale consiste en la finalisation de l'IndissPlus Launcher, une application destinée aux instructeurs pour automatiser la mise en place des sessions de formation sur simulateur, en succédant à l'OTS Manager existant.

Le travail s'est articulé autour de deux axes. D'une côté, l'amélioration du code existant, via la correction de bugs, l'application des principes du Clean Code et la mise en place de tests unitaires. D'un autre côté, le développement de nouvelles fonctionnalités, notamment la pause et l'arrêt de l'exécution, la validation de séquences, ainsi que la conception et le développement complets du mode exécution. Ce dernier permet à l'instructeur de suivre en temps réel l'avancement d'une séquence, d'identifier rapidement les erreurs et d'exporter un rapport d'exécution.

L'application est développée en C# avec le framework .NET 4.8.1, selon une architecture MVVM sous WPF.

Mots clés :

Développement logiciel, C#, MVVM, WPF

Developing and Finalizing the IndissPlus Launcher, a Remote Execution Application for Automating Operator Training Sessions on Simulators

Lili Goy

abstract: In the process industry, instructors running operator training sessions must manually configure every computer before each session. This process is quite time-consuming and can often lead to errors. To address this, Corys initiated the development of the IndissPlus Launcher, a modernized version of an existing solution, the OTS Manager. The object of my work was to finalize this application which automates the launch of action sequences across multiple remote computers. The work was split into two main phases. First the refactoring of the existing codebase by applying Clean Code principles, introducing the Observer pattern to centralize target list management, and adding unit tests for the launcher engine. Second, the development of new features such as the pause and stop commands, the sequence validation and most importantly, an execution mode for the application. The final application allows instructors to monitor sequence executions in real time, to quickly spot, identify and correct errors, from both a step-by-step and a per-computer view. This project contributes to reducing the setup time of training sessions by automating it and reducing the possible errors.

Keywords: Operator Training Simulator, C#, MVVM, WPF